Laboratorio 8 - Teoría de la Computación

Respuestas teóricas

Dally Ramírez - Carné 241035

Octubre 2026

En todo el documento n/2, n/3 en el código representan división entera, como en C. Las mediciones mencionadas son las de la parte b (ver README.md).

① Parte a

```
for (i = n/2; i <= n; i++)
    for (j = 1; j+n/2 <= n; j++)
        for (k = 1; k <= n; k = k*2)
            counter++;
```

Ciclo externo. i va de $\lfloor n/2 \rfloor$ hasta n, entonces da $n - \lfloor n/2 \rfloor + 1 = \lceil n/2 \rceil + 1$ iteraciones.

Ciclo del medio. La condición $j + \lfloor n/2 \rfloor \leq n$ es lo mismo que $j \leq n - \lfloor n/2 \rfloor = \lceil n/2 \rceil$. Como j empieza en 1, da $\lceil n/2 \rceil$ iteraciones. No depende de i.

Ciclo interno. k toma los valores 1, 2, 4, ..., 2^m mientras $2^m \leq n$, o sea $m \leq \log_2 n$. Da

$\lfloor \log_2 n \rfloor + 1$ iteraciones.

Total. Como los tres ciclos son independientes, se multiplican:

$$T(n) = (\lceil n/2 \rceil + 1) \cdot \lceil n/2 \rceil \cdot (\lfloor \log_2 n \rfloor + 1)$$

Simplificación. Para $n \geq 2$:
Cota superior: $\lceil n/2 \rceil \leq n$, $\lceil n/2 \rceil + 1 \leq 2n$ y $\lfloor \log_2 n \rfloor + 1 \leq 2 \log_2 n$, entonces $T(n) \leq 4n^2 \log_2 n$.
Cota inferior: $\lceil n/2 \rceil \geq n/2$ y $\lfloor \log_2 n \rfloor + 1 \geq \log_2 n$, entonces $T(n) \geq (1/4)\, n^2 \log_2 n$.
Por lo tanto

$$\boxed{T(n) = \Theta(n^2 \log n)} \Rightarrow T(n) = O(n^2 \log n).$$

Comparación con las mediciones. Según la fórmula,

$$T(10000) \,/\, T(1000) = \frac{5001 \cdot 5000 \cdot 14}{501 \cdot 500 \cdot 10} \approx 139.7.$$

El tiempo medido subió de 0.0376 s a 5.34 s, un factor de 142. Es un poco más de 100 por el factor log n (de 10 a 14).

② Parte a

```
if (n <= 1) return;
for (i = 1; i <= n; i++)
    for (j = 1; j <= n; j++) {
        printf("Sequence\n");
        break;
    }
```

Caso $n \leq 1$. La función retorna de inmediato: costo constante.

Ciclo externo. i va de 1 a n: n iteraciones.

Ciclo interno. Empieza con j = 1; como $1 \leq n$, entra, imprime y el break sale del ciclo. Nunca llega a j = 2, así que el ciclo interno hace exactamente 1 iteración sin importar n.

Total.

$$T(n) = \sum_{i=1}^{n} 1 = n \Rightarrow T(n) = \Theta(n) = O(n).$$

Aunque hay dos ciclos anidados, no es $O(n^2)$: lo que importa es cuántas veces se ejecuta realmente el ciclo interno.

Comparación con las mediciones. Cada vez que n

se multiplicó por 10, el tiempo se multiplicó por aproximadamente 10 (por ejemplo, de 0.0105 s a 0.1057 s), que es el comportamiento lineal esperado.

③ Parte a

```
for (i = 1; i <= n/3; i++)
    for (j = 1; j <= n; j += 4)
        printf("Sequence\n");
```

Ciclo externo. i va de 1 a $\lfloor n/3 \rfloor$: $\lfloor n/3 \rfloor$ iteraciones.
Ciclo interno. j toma los valores 1, 5, 9, ..., 1 + 4t ≤ n, o sea $t \leq (n - 1)/4$. Da

$$\left\lfloor \frac{n-1}{4} \right\rfloor + 1 = \lceil n/4 \rceil \text{ iteraciones.}$$

Total.

$$T(n) = \lfloor n/3 \rfloor \cdot \lceil n/4 \rceil \approx \frac{n}{3} \cdot \frac{n}{4} = \frac{n^2}{12}$$

Simplificación. Para $n \geq 4$: $\lfloor n/3 \rfloor \leq n/3$ y $\lceil n/4 \rceil \leq n/4 + 1 \leq n/2$, entonces $T(n) \leq n^2/6$. También $\lfloor n/3 \rfloor \geq n/6$ y $\lceil n/4 \rceil \geq n/4$, entonces $T(n) \geq n^2/24$. Por lo tanto

$$T(n) = \Theta(n^2) = O(n^2).$$

Las constantes 1/3 y el paso de 4 solo cambian la constante (1/12), no el orden.

Comparación con las mediciones. Según la fórmula, cada vez que n se multiplica por 10, T(n) se multiplica por 100. Medido: de 0.0060 s a 0.6103 s (factor 101) y de 0.6103 s a 61.11 s (factor 100.1).

(4) Mejor, promedio y peor caso

Interpretación del enunciado. El enunciado dice "Búsqueda Lineal" pero entre paréntesis menciona Linear Search, Binary Search y Quick Sort. Se analizan los tres algoritmos. En todos se cuenta el número de comparaciones sobre un arreglo de tamaño n.

Linear Search

Recorre el arreglo desde la posición 0 hasta encontrar x o llegar al final.
Mejor caso: x está en la primera posición. Se hace 1 comparación: $\Theta(1)$.
Peor caso: x está en la última posición o no está. Se comparan los n elementos: $\Theta(n)$.

Caso promedio: suponiendo que x está en el arreglo y que cada posición es igual de probable (probabilidad 1/n), encontrarlo en la posición i cuesta i comparaciones:

$$\frac{1}{n}\sum_{i=1}^{n} i = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2} \Rightarrow \Theta(n).$$

En promedio revisa la mitad del arreglo, pero sigue siendo lineal.

Binary Search

Requiere un arreglo ordenado. Compara x con el elemento del medio y descarta la mitad que no puede contenerlo.

Mejor caso: x es el elemento del medio. 1 comparación: $\Theta(1)$.

Peor caso: cada iteración deja la mitad de elementos: n, n/2, n/4, ..., 1. Después de k iteraciones quedan $n/2^k$ elementos, y el proceso termina cuando $n/2^k < 1$, es decir con $k = \lfloor \log_2 n \rfloor + 1$ iteraciones: $\Theta(\log n)$.

Caso promedio: con $n = 2^k - 1$ elementos, 1 elemento se encuentra con 1 comparación, 2 con 2, 4 con 3 y en general 2^{d-1} elementos necesitan d comparaciones. Si cada elemento es igual de probable:

$$\frac{1}{n}\sum_{d=1}^{k} d \cdot 2^{d-1} = \frac{(k-1)2^k + 1}{n}$$

$$= (k-1) + \frac{k}{n} \approx \log_2(n+1) - 1 \Rightarrow \Theta(\log n).$$

El promedio es solo una comparación menos que el peor caso, porque la mitad de los elementos están en el último nivel.

Quick Sort

Elige un pivote, particiona el arreglo (n - 1 comparaciones) en los menores y los mayores al pivote, y ordena recursivamente cada parte. Si el pivote queda en la posición q:

$$T(n) = T(q) + T(n - q - 1) + (n - 1).$$

Mejor caso: el pivote siempre queda en el medio, así que las dos partes tienen tamaño n/2:

$$T(n) = 2T(n/2) + cn.$$

Por el Teorema Maestro ($a = 2$, $b = 2$, $f(n) = n = n^{\log_2 2}$) es $\Theta(n \log n)$. Intuitivamente, hay $\log_2 n$ niveles de recursión y en cada uno se hace trabajo cn.

Peor caso: el pivote siempre es el mínimo o el máximo. Por ejemplo, un arreglo ya ordenado con el último elemento como pivote. Una parte

queda vacía y la otra con n - 1 elementos:

$$T(n) = T(n-1) + (n-1) = \sum_{k=1}^{n} (k-1)$$

$$= \frac{n(n-1)}{2} \Rightarrow \Theta(n^2).$$

Caso promedio: con una permutación aleatoria, el pivote tiene la misma probabilidad 1/n de quedar en cada posición q = 0, ..., n - 1:

$$T(n) = (n-1) + \frac{2}{n} \sum_{q=0}^{n-1} T(q).$$

Multiplicando por n y restando la misma ecuación para n - 1:

$$nT(n) - (n-1)T(n-1) = 2(n-1) + 2T(n-1)$$

$$\Rightarrow nT(n) = (n+1)T(n-1) + 2(n-1).$$

Dividiendo entre n(n + 1) y usando

$$\frac{2(k-1)}{k(k+1)} = \frac{4}{k+1} - \frac{2}{k},$$

la suma telescópica con T(1) = 0 da

$$T(n) = 2(n+1)H_n - 4n, \quad H_n = \sum_{k=1}^{n} \frac{1}{k} \approx \ln n.$$

Entonces $T(n) \approx 2n \ln n \approx 1.39\, n \log_2 n$, es decir $\Theta(n \log n)$. (Comprobación: T(2) = 1 y T(3) = 8/3,

igual que con la recurrencia.)

Resumen

Algoritmo	Mejor caso	Caso promedio	Peor caso
Linear Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Binary Search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Quick Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$

⑤ Verdadero o falso

Se asume, como es usual, que las funciones son positivas para n suficientemente grande.

a) Si $f = \Theta(g)$ y $g = \Theta(h)$, entonces $h = \Theta(f)$.

Verdadero

Por definición existen constantes positivas tales que:

$$c_1\, g(n) \le f(n) \le c_2\, g(n) \quad (n \ge n_1),$$
$$c_3\, h(n) \le g(n) \le c_4\, h(n) \quad (n \ge n_2).$$

Para $n \ge \max(n_1, n_2)$, sustituyendo la segunda en la primera:

$$c_1\, c_3\, h(n) \le f(n) \le c_2\, c_4\, h(n).$$

Despejando h(n):

$$\frac{1}{c_2\, c_4} f(n) \le h(n) \le \frac{1}{c_1\, c_3} f(n),$$

con constantes positivas, así que $h(n) = \Theta(f(n))$.
Esto muestra que Θ es transitiva y simétrica.

b) Si $f = O(g)$ y $g = O(h)$, entonces $h = \Omega(f)$.

Verdadero

Existen constantes positivas con

$$f(n) \leq c_1\, g(n) \quad (n \geq n_1),$$
$$g(n) \leq c_2\, h(n) \quad (n \geq n_2).$$

Para $n \geq \max(n_1, n_2)$: $f(n) \leq c_1\, c_2\, h(n)$, o sea $f = O(h)$ (transitividad). Despejando:

$$h(n) \geq \frac{1}{c_1\, c_2} f(n),$$

que es justo la definición de $h(n) = \Omega(f(n))$.

c) El tiempo de A(n) es $\Theta(n^2)$.

Falso

```
def A(n):
    atupla = tuple(range(0, n))
    S = set()
    for i in range(0, n):
        for j in range(i + 1, n):
            S.add(atupla[i:j])
```

Es cierto que el cuerpo del ciclo se ejecuta

$$\sum_{i=0}^{n-1} (n - 1 - i) = \frac{n(n-1)}{2} = \Theta(n^2) \text{ veces,}$$

pero cada iteración no cuesta O(1):
atupla[i:j] crea una tupla nueva copiando j - i elementos: costo $\Theta(j - i)$.
S.add(...) tiene que calcular el hash de esa tupla, y el hash de una tupla se calcula recorriendo todos sus elementos: otra vez $\Theta(j - i)$. Como todas las rebanadas son distintas, no hay comparaciones adicionales por duplicados. Además, tuple(range(0, n)) cuesta $\Theta(n)$, que no afecta. Entonces:

$$f(n) = \Theta\left(\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} (j - i)\right).$$

Con $d = j - i$ y $m = n - 1 - i$:

$$\sum_{i=0}^{n-1} \sum_{d=1}^{n-1-i} d = \sum_{m=0}^{n-1} \frac{m(m+1)}{2}$$

$$= \frac{(n-1)\, n\, (n+1)}{6} = \Theta(n^3).$$

Por lo tanto $f(n) = \Theta(n^3)$. Como n^3 no es $O(n^2)$, la afirmación $f(n) = \Theta(n^2)$ es falsa. (Además el set termina guardando $\Theta(n^3)$ elementos en total, así que la memoria también es cúbica.)

Verificación experimental. Midiendo A(n) en Python, al duplicar n (200, 400, 800) el tiempo se multiplicó por aproximadamente 7.5. Si fuera $\Theta(n^2)$ se multiplicaría por 4; con $\Theta(n^3)$ se espera 8.